

어빌리티 시스템이란 무엇인가

코드를 짜기 전에 개념부터. 게임 속 모든 "발동되는 행위"를 하나의 문법으로 다루는 틀에 대하여.

한 문장 정의 — 어빌리티 시스템은 평타·스킬·보스 패턴·함정처럼 게임 안에서 "무언가가 발동되어 효과를 일으키는 모든 것"을, 캐스터마다 따로 짜지 않고 **하나의 공통 문법(누가·무엇을·어떻게·언제)**으로 기술하고 실행하는 틀이다.

목적

개념 재정립 (설계 이전 단계)

대상 독자

구현 착수 전의 나 자신

범위

플레이어 · 일반몹 · 보스 공통

01 왜 필요한가 — "무대뽀"의 정체

시스템이 없다는 건 코드가 없다는 뜻이 아니라, "발동"에 **정해진 형태가 없다는 뜻이다.**

게임에는 "발동되는 행위"가 수십 가지다. 캐릭터의 평타, Q 스킬, 보스의 소환 패턴, 탱크의 포격, 바닥의 장판. 이것들은 겉보기엔 전부 달라 보이지만, 속을 뜯어보면 **똑같은 골격**을 갖는다 — 누군가가, 무언가를, 어떤 상황에서, 발동해서, **효과**를 낸다.

시스템 없이 만들면 이 골격이 코드 곳곳에 **제각각 다시 태어난다.** "총을 쏜다"가 AR몹에 한 번, 플레이어에 한 번, 보스에 한 번 — 개념은 하나인데 구현이 세 별이다. 새 스킬은 새 클래스, 값 하나 바꾸려면 컴파일. 이게 "계획 없이 무대뽀로 했다"의 정확한 정체다: **같은 개념을 반복해 짜면서, 그 개념에 이름을 붙여주지 않은 상태.**

그래서 어빌리티 시스템이 하는 일

이 반복되는 골격에 **이름과 형태를 부여**한다. 한 번 형태를 정하면, 이후 모든 발동은 "코드를 새로 짜는 일"이 아니라 "정해진 칸을 채우는 일(데이터)"이 된다. 프로그래머 병목이 사라지고, 디자이너가 스킬을 만든다.

※ 이 문서는 "무엇을 이미 구현했는가"(이관 기록)가 아니라 "**이 시스템이 개념적으로 무엇인가**"를 다룬다. 구현 세부는 [README.md](#) · [AbilitySystem_Portfolio.md](#) 참고.

02 개념의 뼈대 — 다섯 개의 질문

어떤 발동이든 이 다섯 조각으로 쪼갤 수 있다. 각 조각이 시스템의 한 부품이 된다.

WHO · 누가

캐스터 (Caster)

발동하는 주체. 플레이어든 보스든 잡몹이든, "나는 어빌리티를 쓸 수 있는 존재"라는 공통 신분을 가진다.

→ [IAbilityCaster](#)

WHAT · 무엇을

어빌리티 (Ability)

발동할 "그 스킬"의 정의. 코스트·쿨다운·캐스트 타임과, 무슨 효과들이 어떤 순서로 나가는지의 목록.

→ [AbilityData \(SO\)](#)

HOW · 무슨 일이

이펙트 (Effect)

실제로 일어나는 일 하나하나. 데미지 주기, 투사체 쏘기, 장판 깔기, 연출 재생 — 재사용되는 조립 블록.

→ [IEffect](#)

WHOM · 어떤 상황

컨텍스트 (Context)

이번 한 번의 발동에만 해당하는 정보. 시전자의 스탯, 겨냥한 대상, 착탄 좌표, 총구 위치.

→ [AbilityContext](#)

WHEN · 지금 써도 되나

게이팅 (Gating)

발동을 허락하거나 막는 조건. 쿨다운이 안 돌아나 · 코스트가 있나 · "기절 중이라 발동 불가" 같은 상태 조건. 코드의 `if` 가 아니라 데이터로 표현하는 것이 핵심.

→ [AbilityRunner \(쿨다운\)](#) + [GameplayTag \(상태 조건\)](#)

이 다섯이 전부다

새로운 스킬을 상상할 때 "이건 시스템으로 표현 가능한가?"를 묻는 대신, 이 다섯 칸을 채울 수 있는가를 물으면 된다. 채워지면 데이터로 만들 수 있고, 안 채워지면 새 **블록(Effect)**이 하나 필요하다는 신호다.

03 비유로 잡는 직관 — 주방

추상 타입들이 헛갈릴 때, 이 대응표 하나면 관계가 다시 선명해진다.

어빌리티 시스템은 **주방**과 똑같이 생겼다. 가장 헛갈리는 지점 — "정의(데이터)"와 "실행(런타임)"의 차이 — 이 비유가 단번에 정리해 준다.

주방에서는	시스템에서는	성질
레시피 카드	어빌리티 / 이펙트 정의 (AbilityData, EffectData)	종이 한 장. 100번 요리해도 안 변함. 모두가 공유.
조리 단계 (썰기·볶기·끓이기)	이펙트 블록 (DealDamage, SpawnProjectiles...)	재사용되는 동작. 여러 레시피가 같은 단계를 씀.
요리사	캐스터 (IAbilityCaster)	레시피대로 만드는 주체. 누구든 같은 방식으로.
이번 주문표 (3번 테이블, 덜 맵게)	컨텍스트 (AbilityContext)	이 한 번에만 해당. 대상·좌표·스탯.
실제로 나온 접시	런타임 이펙트 인스턴스 (IEffect)	매번 새것. 만들어지는 도중 상태가 있음.
재료 재고 · 화구 점유	코스트 · 쿨다운 · 상태 태그	지금 만들어도 되는지의 조건.

이 비유가 지켜주는 단 하나의 규칙

레시피 카드에 **낙서하지 마라**. 요리가 짜다고 레시피 종이의 소금량을 볼펜으로 고치면, 다음 요리사도, 내일의 나도 영영 그 값으로 만든다. → 이게 **"SO 필드를 런타임에 바꾸지 마라"**의 진짜 이유다. 바꾸고 싶으면 접시(런타임 인스턴스) 위에서 바꾸거나, 레시피 카드를 **한 장 더** 만든다.

04 핵심 사상 — "상속"이 아니라 "조합"

스킬 하나 = 클래스 하나였던 세계에서, 스킬 하나 = 블록의 목록인 세계로.

어빌리티 시스템의 심장은 이 한 번의 관점 전환이다. 스킬을 **덩어리**로 보지 않고 **블록의 배치**로 본다. 레고와 같다 — 새 모델은 새 부품을 깎는 게 아니라, 있는 부품을 새로 배치하는 것.

이전 · 상속

스킬 = 클래스 하나

- "불꽃 3연발 + 화상"을 만들려면 **새 클래스**를 짤다
- 거의 같은 스킬도 값 하나 다르면 클래스가 갈라짐
- 조합 불가 — 데미지·상태이상·연출이 한 덩어리로 굳음
- 새 스킬 = 새 코드 + 재컴파일 (프로그래머만)

```
class FireTripleBurnSkill :  
    SkillBase { ... 전부 다시 ... }
```

현재 · 조합

스킬 = 블록의 목록

- 있는 블록을 **배치**만 한다 — 새 코드 0
- 값이 다르면 같은 블록의 asset을 하나 더 (Count 3 vs 1)
- 데미지 + 화상 + 연출을 **자유롭게 엮음**
- 새 스킬 = asset 편집 (디자이너 가능)

```
Repeat(3) { SpawnProjectiles(1) }  
+ ApplyStatus(Burn)  
+ SpawnVFX(Muzzle)
```

언제 새 블록(EFFECT)이 필요한가 — 값/로직/상태 3축

값이 다를 뿐이면(5발 vs 1발) → asset을 하나 더. **로직**이 근본적으로 다르면(투사체 vs 지면 장판) → 새 Effect 클래스. 발동 도중 **기억할 상태**가 있으면(도트의 경과시간) → 런타임 인스턴스로 분리. 이 세 갈래가 "언제 코드를 쓰고 언제 데이터로 끝내는가"의 판단 기준이다.

05 한 번의 발동이 흐르는 길

입력에서 효과까지, 모든 캐스터가 똑같은 한 줄기로 흐른다. 이 단일 경로가 시스템의 존재 이유다.

1

발동 신호

애니메이션 이벤트 · 플레이어 입력 · 타임라인 시그널 중 하나가 "지금, 이 어빌리티" 하고 방아쇠를 당긴다. 타이밍의 주인은 발동자마다 다르지만, 도착지는 하나다.

2

캐스터가 어빌리티를 지목

캐스터가 자기 컨텍스트(스탯·총구·대상)를 꾸려서 러너에게 넘긴다. "이 어빌리티를, 이 상황으로 써 줘."

3

게이트 — 지금 써도 되나?

쿨다운이 돌아나 · 코스트가 되나 · 요구 태그를 갖고 차단 태그가 없나. 하나라도 실패하면 여기서 조용히 멈춘다(쿨다운도 소모 안 함).

4

(캐스트타임) 시전 대기

채널링이 있으면 그만큼 기다린다. 이 동안 "시전 중" 태그가 붙어 다른 발동을 막을 수 있다. 중간에 취소되면 태그는 반드시 풀린다.

5

이펙트를 순서대로 실행

어빌리티에 얹힌 블록들이 위에서 아래로 흐른다. 투사체 스폰 → 총구 화염 → 데미지 → 상태이상 부여... 각 블록은 자기 일만 안다.

6

결과 적용

데미지 파이프라인, 상태이상 등록, 연출 재생. 주의 — 투사체 데미지는 "발동 시점"이 아니라 "명중 시점"에 확정된다(아래 §07).

왜 굳이 "하나의 길"인가

플레이어·잡몹·보스가 각자 다른 길로 발동하면, 쿨다운·태그·취소 안전을 세 번 구현하고 세 번 버그를 고쳐야 한다. 길을 하나로 합치면 그 모든 규칙이 한 곳(러너)에만 살고, 새 캐스터는 그 길에 올라타기만 하면 된다.

06 가장 어려운 축 — "상태"와 두 개의 시간

발동은 쉽다. 어려운 건 "발동이 끝난 뒤에도 남는 것"이다.

데미지 한 방은 발동하고 끝난다. 하지만 화상 5초, 공격력 버프, 기절 — 이것들은 발동이 끝나도 **붙어서 시간이 흐른다**. 어빌리티 시스템이 초심자에게 가장 헷갈리는 지점이 바로 이 "두 개의 시간"을 구분하는 것이다.

⚡ 순간 (Instant)

발동하는 즉시 일어나고 끝나는 것. 이펙트 블록이 실행되면 그 자리에서 완결된다. 상태를 남기지 않는다.

DealDamage · SpawnProjectiles ·
SpawnVFX

⌚ 지속 (Duration)

발동은 "부착"만 하고, 그 뒤로 **맞은 쪽에 붙어** 스스로 시간이 흐르는 것. 도트 데미지, 버프, 기절 창.

ApplyStatus(Burn 5s) · Defense -30%
6s

지속 상태는 누가 소유하는가 — "맞은 쪽"이다

여기서 결정적인 개념 하나. 화상은 **때린 사람**이 아니라 **맞은 사람**에게 붙는다. 그래서 지속 상태를 tick(갱신·만료)하는 주인은 발동자가 아니라 **대상**이다. 이펙트는 "불을 붙이는 일"만 하고 손을 떼다. 그 뒤 타들어 가는 건 대상의 몫(StatusSystem). 때린 사람이 죽어도 화상은 계속 타야 하니까.

또 하나의 상태 — 조건 태그 (발동 가능/불가)

"기절 중엔 아무것도 못 쓴다", "가드 중엔 특정기만" 같은 건 **효과**가 아니라 **조건**이다. 이걸 태그로 표현한다. 중요한 건 이 조건 축이 **FSM(Idle/Move/Attack...)**과 **직교(병렬)**한다는 것 — 기절은 어느 모드에도 겹칠 수 있어서 FSM 상태로 넣으면 조합이 폭발한다. 그래서 FSM은 그대로 두고, 병렬 조건만 태그가 담당한다.

상태를 세 통에 나눠 담아라

순간 효과 = 이펙트가 즉시 처리

지속 효과 = 대상의 StatusSystem이 tick

발동 조건 = 태그(FSM과 병렬)

이 세 통을 섞으면 무대뽀가 되고, 나누면 시스템이 된다.

07 무대뽀 방지 체크리스트

구현에 손대기 전에 이 질문들에 먼저 답이 서 있어야 한다. 계획이란 곧 이 답들이다.

Q1 발동 주체의 공통 계약은 무엇인가?

모든 캐스터가 공유하는 최소한의 약속. **답:** "어빌리티를 발동할 수 있다"는 신분 하나. 발동 진입점(입력/애니/타임라인)은 달라도 된다.

Q2 값이 다른가, 로직이 다른가, 상태가 다른가?

새 요구가 올 때마다 이 3축으로 분류. **값**→asset 추가 · **로직**→새 Effect · **상태**→런타임 인스턴스. 분류를 건너뛰면 클래스가 폭발한다.

Q3 데이터(불변)와 런타임(가변)의 경계는 어디인가?

레시피 카드 / 나온 접시. 공유·불변은 SO, 매번 새것·상태 있음은 인스턴스. **SO 필드는 런타임에 절대 안 바꾼다.**

Q4 발동 타이밍은 누가 소유하는가?

플레이어=애니메이션 이벤트(프레임 정확도), 보스=타임라인 시그널, 잡몹=BT/애니. 타이밍의 주인은 달라도 도착지는 러너 하나.

Q5 데미지는 언제 확정되는가 — 발동 시점? 명중 시점?

투사체는 **명중 시점**. 계산은 발동자 스탯으로 하되, 값을 스냅샷 패킷으로 **투사체가 배달한다**. 발동자가 죽어도(풀 반환) 안전.

Q6 지속 상태는 누가 소유하고 누가 tick 하는가?

대상이 소유하고 tick. 이펙트는 부착만. 때린 사람과 상태의 수명을 분리해야 죽음·풀 반환에서 안전하다.

Q7 발동 가능/불가 조건은 코드인가 데이터인가?

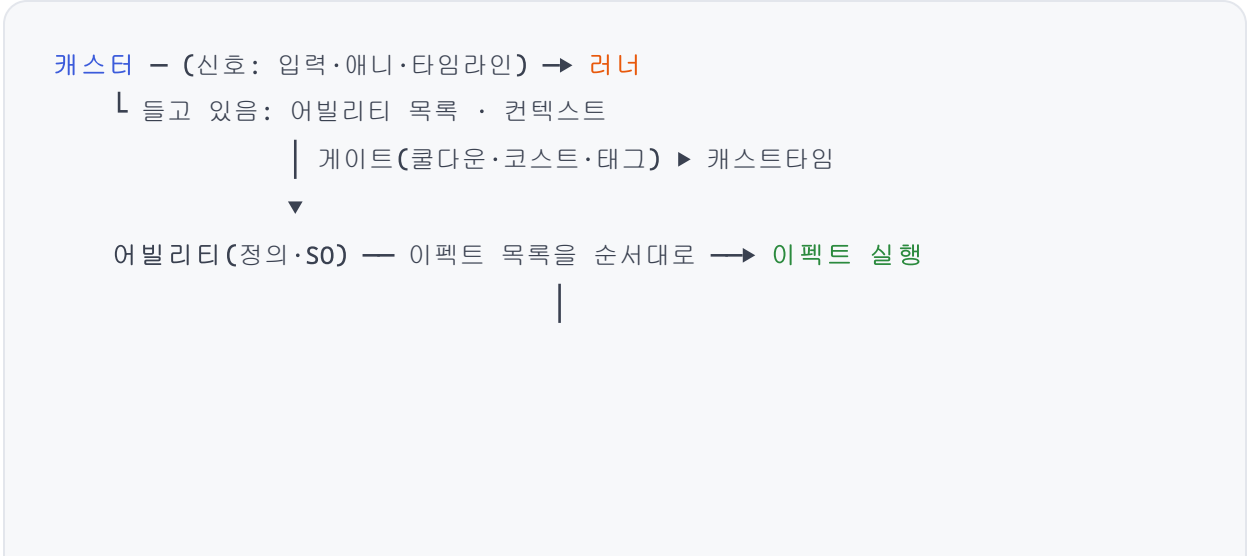
데이터(태그). "기절 중 불가"를 `if` 로 흩뿌리면 스킬마다 재구현. Required/Blocked 태그로 러너 한 곳에서 게이팅.

08 개념을 우리 코드에 대입

위에서 세운 개념 어휘가, 실제 프로젝트에서 어떤 타입으로 살아 있는지의 대응표.

개념 (이 문서)	우리 구현	역할
누가 발동하나	<code>IAbilityCaster</code> — <code>BaseCharacter</code> · <code>BaseMonsterController</code> · <code>BossMonsterController</code>	캐스터 신분
무엇을 발동하나	<code>AbilityData</code> (<code>ScriptableObject</code>)	스킬 정의
발동 게이트·타이밍	<code>AbilityRunner</code> — 쿨다운/캐스트타임 소유, <code>TryCast</code>	단일 경로
이번 상황	<code>AbilityContext</code> — 스탯·대상·착탄점·충구·태그	발동 문맥
조립 블록 (정의)	<code>EffectData</code> (추상 SO) → <code>CreateRuntime()</code>	레시피 단계
조립 블록 (실행)	<code>IEffect</code> — <code>DealDamage</code> · <code>SpawnProjectiles</code> · <code>AreaStrike</code> · <code>RepeatEffect ...</code>	런타임 동작
지속 효과	<code>StatusRunner</code> · <code>ApplyStatModifier</code> / <code>ApplyStatusTag</code>	대상이 tick
발동 조건 (병렬 상태)	<code>GameplayTagSO</code> · <code>GameplayTagContainer</code> (<code>Required/Blocked/Grants</code>)	Lite GAS

개념 지도





순간→ 데미지·투사체·연출



지속→ 대상 **StatusSystem**이 tick

09 개념적 경계선 — 자주 혼동하는 것들

이 네 개를 헷갈리면 시스템이 다시 무대뽀로 무너진다. 경계를 또렷하게.

혼동	경계
"발동 = 데미지"	아니다. 발동은 투사체를 쏘는 것 까지고, 데미지는 명중 시점 에 별도로 확정된다. 이 둘을 붙여 놓으면 빗나간 총알도 데미지를 주거나, 발동자가 죽으면 데미지가 사라진다.
"태그 = FSM 상태"	아니다. FSM은 배타적 모드 (한 번에 하나: Idle이거나 Attack이거나). 태그는 병렬 조건 (기절이면서 동시에 표식 걸림). 합치면 상태 조합이 폭발한다.
"SO 값을 바꾸면 되지"	안 된다. SO는 공유되는 레시피 카드 . 런타임에 고치면 모든 인스턴스가 오염되고, 에디터에선 플레이 종료 후에도 값이 영구히 남는다. 변형은 asset을 하나 더 만들거나 스탯을 곱해 읽는다.
"반복 안에 반복"	조합엔 함정이 있다. Repeat(5) 안의 SpawnProjectiles도 Count 5면 25발 이 나간다. 반복의 책임은 한 곳(Repeat) 에만 둔다. 조합의 자유엔 "책임의 단일 소재지"라는 규율이 따라온다.

이 문서를 한 문장으로

어빌리티 시스템은 "발동되는 모든 것"을 다섯 질문(누가·무엇을·어떻게·어떤 상황·언제)으로 쪼개고, 정의(레시피)와 실행(접시)을 분리한 채, 스킬을 상속이 아닌 조합으로 짓는 틀이다. 이 개념이 서면, 그다음 구현은 "칸을 채우는 일"이 된다.